

Introduction :-

Document Editor :-

Features :-

- Text , images , - can be inserted

Our Editor should be scalable :-

In future :- we can insert

- Videos, Tables, font, Tabs, new line.

Approach to follow while designing LLD :-

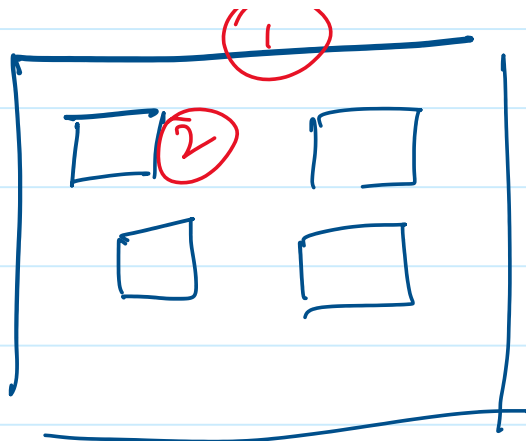
1. Top-down Approach -

- first we make Top most object i.e main object of our application and then we create small-small object inside it.

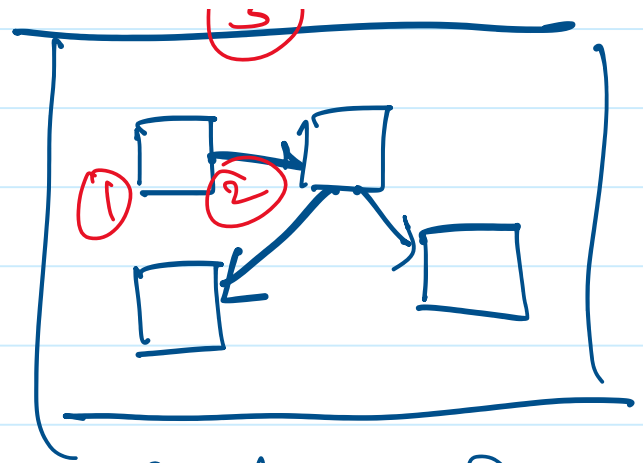
2. Bottom-up Approach

- first we create small object and then create dependency b/w them and then move to main object





Top-down



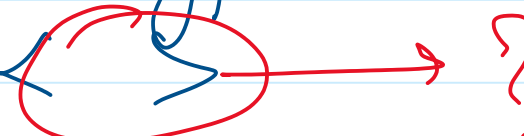
Bottom-up

Document Editor :-

In our Document Editor we have text & image.

We can store them in list. But to maintain the order of text & image we put them in list

Now, which Datatype to use

List 

for text → String

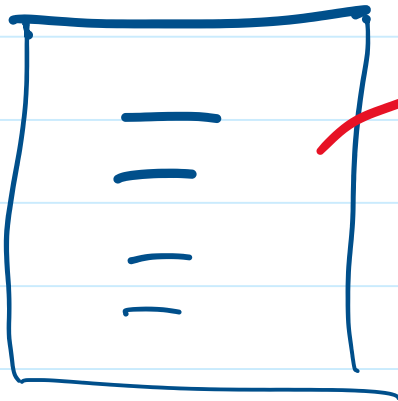
for image → we have to use same Datatype as both are in same list.

for image we can store the path of image and later we can render them on Docs

```
DocumentEditor  
{  
    Vector<String> elements;  
}
```

```
addText(String text);  
addImage(String Path);  
renderDocument();  
saveToFile();
```

renderDocument() :-



This is a frontend where user is writing the document



Button when the user click on this he will see the whole document how it looks

So, this method loop through the elements list and render the whole document.

We can give the rendered result in String.

saveToFile() :-

There will be one more button to save the document to file or DB.

save the document to file or DB.

Code :- see Notes...

Note :- In LLD your logic in functions or method is not important.
The more important is how much your design is optimized.

Problems in our last Design:-

(i) - SRP - X Not followed.

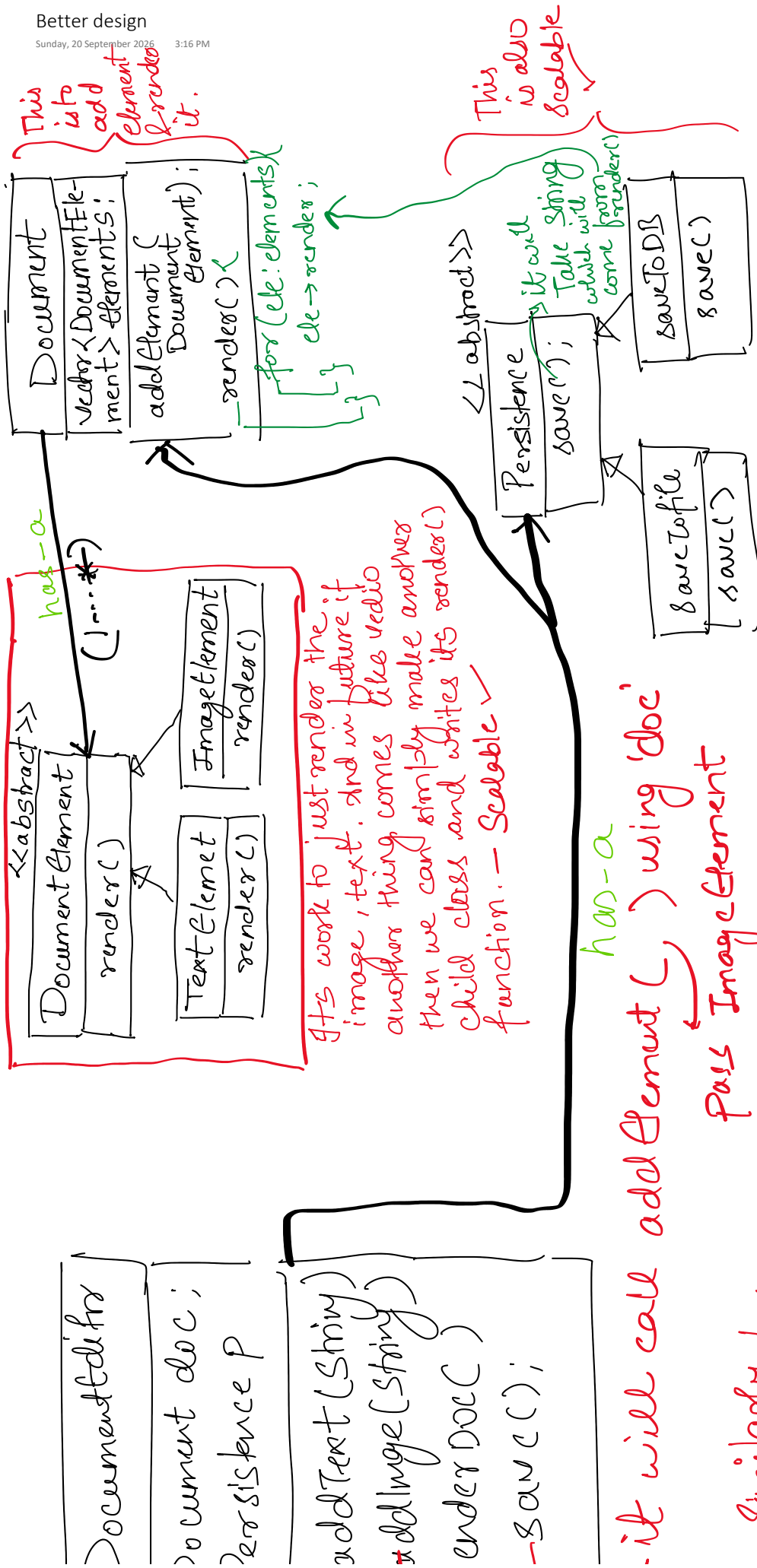
b/c Document Editor is doing all the thing.

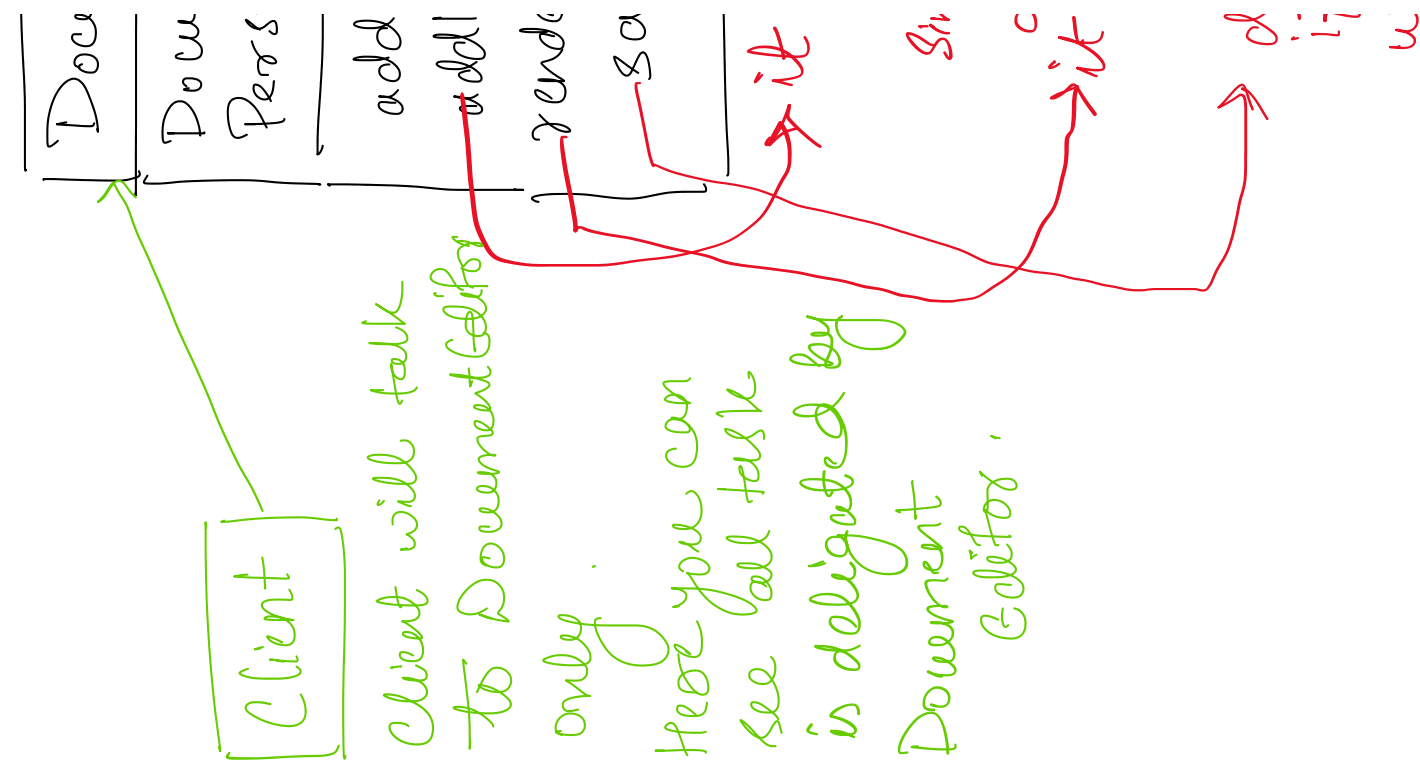
(ii) - OCP - X Not followed.

b/c in future when you put new feature then you will edit the same class by adding new method like - for video, etc...

(iii) - AIP, DIP, ISP X Not Available

b/c only single class is there.





Code — see Notes....

Problem in better design

Sunday, 20 September 2026 4:01 PM

SRP - Followed ✓ - Every class has their own work.

OCP - followed ✓ - Any new features can be added without modifying existing classes. ✓
like adding new Element - "VideoElement"
or new Storage
like - "SaveToCassandra"

LSP - followed ✓ - All parent class are replaceable by their child class easily. like
"DocumentElement" is easily replaceable by "TextElement"
or "ImageElement"
Same for "Persistence"

ISP - followed ✓ - There is no such method in Parent class which forcefully child has to implement.

to implement.

DIP - followed ✓ - All High level Model are connected with low-level module via interface.
Ex -

Document is not directly interacting with "TextElement" or "ImageElement" but it is connected via Interface "DocumentElement".
Similarly "DocumentEditor" is connected with Interface "Persistence" - not directly to "saveToFile" or "saveToDB".

Small Improvement:-

"Document Editor" class is not directly violating the SRP but

if in future we have to 'render()' from "Document" class then from "DocumentEditor" also we have to remove the "renderDoc()".

So,

So,
it doesn't have more than one reason
to change but it knows more than
one reason to get change.

So, is it following SRP?

↓
Answer
can
be "yes" or "No"

it means, "DocumentEditor" doesn't know
how to renderDoc() or save() b/c it
is delegating its work to other classes
But it has the knowledge of render & save

→ By the logic of delegation you can say
it is following SRP. But by
the logic of knowledge you can say
No.

Final - Best Design — see Notes...

Now, after Getting the Best design
we still have one problem called

we 'still' have one problem called
- Principal of least knowledge

→ it says, a class should only talk to its immediate friend not to friend-of-friend.

But in our best design:-

1. In "DocumentRender" class render() method logic is:-

first it calls getElement() to get list of "DocumentElement"

Then it loops through it and for each "DocumentElement" it calls render() method "DocumentElement" class.

So, "DocumentElement" class is directly talking to "DocumentElement" class which is friend-of-friend

∴ Principal of least knowledge

Breaks X

To solve this:-

Again we move "render()" method in "DocumentRender" class to

in "DocumentRender" class to
"Document" class

↳ But again by doing this SRP
breaks ∞ SIKDII |

Note:-

So, there is always a trade-off
and you have to come to final
Design

So, Nothing is perfect LLD. There
is always a open question